LEMBAR REVISI SIDANG PROPOSAL SKRIPSI

Semester Gasal 2025/2026

Nama : YESTOYA LUMENCHRISTO MINGGUS
NRP : C14220331
Judul Skripsi : Implementasi Mekanisme Server-Authoritative Clock dan Client-Side Interpolation pada Aplikasi Partitur Musik Kolaboratif Real-Time
Tanggal Ujian : 9 Desember 2025 Pukul : 09:30
Penguji : 21011 - Hans Juwiantho, S.Kom., M.Kom.

No	Materi yang di Revisi
1	Sudah pernah melihat masalahnya secara langsung ? di tambahkan di latar belakang "masalah" real nya. Apakah ini di 1 ruangan yang sama ? Rumusan 1 masih how to. harap di ganti

Surabaya, 15 Desember 2025
Dosen Penguji,

Hans Juwiantho, S.Kom., M.Kom.
Nip : 21011

Kontrol Oleh	Keterangan	Paraf
Pembimbing I dan II	Sudah / Belum Perbaikan	

LEMBAR REVISI SIDANG PROPOSAL SKRIPSI

Semester Gasal 2025/2026

Nama : YESTOYA LUMENCHRISTO MINGGUS
NRP : C14220331
Judul Skripsi : Implementasi Mekanisme Server-Authoritative Clock dan Client-Side Interpolation pada Aplikasi Partitur Musik Kolaboratif Real-Time
Tanggal Ujian : 9 Desember 2025 Pukul : 09:30
Penguji : 02030 - Dr. Gregorius Satia Budhi, S.T., M.T.

No	Materi yang di Revisi
1	Bisa dicoba untuk beberpa orang yg bermain di lokasi berbeda.
2	Bisa di teliti apa perlu resinkronisasi dan di menit berapa

Surabaya, 15 Desember 2025
Dosen Penguji,

Dr. Gregorius Satia Budhi, S.T., M.T.
Nip : 02030

Kontrol Oleh	Keterangan	Paraf
Pembimbing I dan II	Sudah / Belum Perbaikan	

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN PETRA

USULAN PROYEK AKHIR

Nama	:	Yestoya Lumenchristo Minggus
NRP	:	C14220331
Program	:	Informatika
Judul proyek akhir	:	Implementasi Mekanisme Server-Authoritative Clock dan Client-Side Interpolation pada Aplikasi Partitur Musik Kolaboratif Real-Time
Pembimbing 1	:	Justinus Andjarwirawan, S.T.,M.Eng.
Pembimbing 2	:	Dr. Andreas Handojo, S.T., M.MT.
Dilaksanakan	:	Genap 2025/2026

Surabaya, 15 Desember 2025
Yang mengusulkan

Yestoya Lumenchristo
Minggus

Pembimbing 1

Justinus Andjarwirawan, S.T.,M.Eng.

Pembimbing 2

Dr. Andreas Handojo, S.T., M.MT.

Koordinator Skripsi

Liliana, S.T., M.Eng Ph.D.

Transkrip

Nama	:	YESTOYA LUMENCHRISTO MINGGUS	Fakultas	:	TEKNOLOGI INDUSTRI
NRP	:	C14220331	Program Studi	:	INFORMATIKA
Tempat/Tgl.Lahir	:	SEMARANG, 03/08/2004	Program	:	INFORMATIKA
Tahun Masuk	:	1-2022/2023	Program Pendidikan	:	S1

Kode	Mata Kuliah	SMT	SKS	Nilai
FD4505	KALKULUS I / CALCULUS I	1-22/23	3	C
DU4122	BAHASA INDONESIA / INDONESIAN	1-22/23	2	A
TF4536	DASAR PEMROGRAMAN / PROGRAMMING BASICS	1-22/23	4	B
TF4205	DASAR SISTEM KOMPUTER / INTRODUCTION TO COMPUTER SYSTEM	1-22/23	2	C+
TF4220	PENGANTAR MANAJEMEN DAN BISNIS / INTRODUCTION TO MANAGEMENT AND BUSINESS	1-22/23	2	A
DU4197	AGAMA DAN HIDUP BERMAKNA / A MEANINGFUL LIFE	1-22/23	4	A
TF4537	KONSEP ALGORITMA / ALGORITHM CONCEPTS	1-22/23	3	B+
DU4101	PANCASILA / STATE IDEOLOGY	2-22/23	2	A
TF4229	BASIS DATA / DATABASE	2-22/23	3	B+
TF4245	MATEMATIKA DISKRIT / DISCRETE MATHEMATICS	2-22/23	3	B+
TF4227	STATISTIKA DASAR / BASIC STATISTICS	2-22/23	3	B
TF4267	KOMUNIKASI INTERPERSONAL / INTERPERSONAL COMMUNICATION	2-22/23	3	B+
TF4253	JARINGAN KOMPUTER / COMPUTER NETWORKS	2-22/23	3	B
TF4235	PEMROGRAMAN BERORIENTASI OBYEK / OBJECT-ORIENTED PROGRAMMING	2-22/23	3	B+
FD4507	ALJABAR LINIER / LINEAR ALGEBRA	1-23/24	3	B+
DU4164	PENDIDIKAN KEWARGANEGARAAN / CIVICS EDUCATION	1-23/24	2	A
TF4504	BAHASA INGGRIS / ENGLISH	1-23/24	2	A
TF4255	REKAYASA PERANGKAT LUNAK / SOFTWARE ENGINEERING	1-23/24	3	B+
TF4270	DESAIN DAN ANALISIS ALGORITMA / DESIGN AND ANALYSIS OF ALGORITHMS	1-23/24	3	B
TF4372	ARSITEKTUR DAN ORGANISASI KOMPUTER / COMPUTER ARCHITECTURES AND ORGANIZATION	1-23/24	3	C
TF4219	STRUKTUR DATA / BASIC STUCTURES	1-23/24	3	B

Kode	Mata Kuliah	SMT	SKS	Nilai
TF4343	TEKNOLOGI WEB / WEB TECHNOLOGIES	1-23/24	3	A
DU4163	ETIKA PROFESI / PROFESSIONAL ETHICS	2-23/24	2	A
TF4327	ANALISIS DAN DESAIN SISTEM INFORMASI / ANALYSIS AND DESIGN OF INFORMATION SYSTEM	2-23/24	3	B+
TF4237	INTERAKSI MANUSIA DAN KOMPUTER / HUMAN-COMPUTER INTERACTION	2-23/24	3	B+
TF4243	SISTEM OPERASI / OPERATING SYSTEM	2-23/24	3	B+
TF4415	GRAFIKA KOMPUTER / COMPUTER GRAPHICS	2-23/24	3	B
TF4409	KECERDASAN BUATAN / ARTIFICIAL INTELLIGENCE	2-23/24	3	B+
TF4247	METODE NUMERIK / NUMERICAL METHODS	2-23/24	2	B+
TF4544	CYBER OPERATIONS / CYBER OPERATIONS	2-23/24	3	B+
FD4508	TECHNOPRENEURSHIP / TECHNOPRENEURSHIP	1-24/25	3	B+
TF4579	WEB FRAMEWORKS AND DEPLOYMENT / WEB FRAMEWORKS AND DEPLOYMENT	1-24/25	3	A
TF4509	CLOUD COMPUTING / CLOUD COMPUTING	1-24/25	3	B+
DU4198	DIGITAL LEADERSHIP / DIGITAL LEADERSHIP	1-24/25	2	A
TF4317	MANAJEMEN PROYEK TEKNOLOGI INFORMASI / INFORMATION TECHNOLOGY PROJECT MANAGEMENT	1-24/25	3	A
TF4590	DATA ANALYTICS ESSENTIALS / DATA ANALYTICS ESSENTIALS	1-24/25	2	A
TF4516	PENGEMBANGAN APLIKASI BERBASIS ANDROID / ANDROID APPLICATION DEVELOPMENT	1-24/25	3	A
TF4507	SISTEM TERDISTRIBUSI / DISTRIBUTED SYSTEMS	1-24/25	3	B
TF4259	METODOLOGI PENELITIAN / RESEARCH METHODOLOGY	2-24/25	3	B+
TF4540	INDUSTRIAL TRAINING / INDUSTRIAL TRAINING	2-24/25	6	B+
TF4261	KERJA PRAKTEK / INTERNSHIP	2-24/25	2	B+
TF4539	PROFESSIONAL DEVELOPMENT / PROFESSIONAL DEVELOPMENT	2-24/25	6	A

Keterangan :

Indeks Prestasi Kumulatif	:	3.48
Jumlah SKS	:	123 SKS

Nilai Huruf	Keterangan	Nilai Bobot
A	Istimewa	4.00
B+	Baik Sekali	3.50
B	Baik	3.00
C+	Cukup Baik	2.50
C	Cukup	2.00
D	Kurang	1.00
E	Sangat Kurang	0.00

PROPOSAL SKRIPSI

Implementasi Mekanisme *Server-Authoritative Clock* dan *Client-Side Interpolation* pada Aplikasi Partitur Musik Kolaboratif *Real-Time*

Oleh :

Yestoya Lumenchristo Minggus NRP: C14220331

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNOLOGI INDUSTRI

UNIVERSITAS KRISTEN PETRA

2025

1. **Judul**

   Implementasi Mekanisme S*erver-Authoritative Clock* dan *Client-Side Interpolation* pada Aplikasi Partitur Musik Kolaboratif *Real-Time*

2. **Latar Belakang**

   Dalam sepuluh tahun terakhir, kemajuan teknologi digital mendorong pertumbuhan di bidang musik, mulai dari aplikasi pemutar musik atau *streaming*, aplikasi yang membantu dalam produksi musik atau DAW *(Digital Audio Workstations)*, sampai platform kolaborasi online yang mempermudah musisi dalam membuat musik termasuk berkolaborasi dengan musisi lain. Perkembangan secara tidak langsung dipercepat oleh pandemi COVID-19, yang mengubah cara bermusik dan memaksa komunitas musisi di seluruh dunia beradaptasi ke lingkungan yang lebih digital (Onderdijk et al., 2021). Menurut NPR (2025), pandemi COVID-19 mempercepat adopsi platform kolaborasi dan pertunjukkan musik online, menjadikan aplikasi digital bagian dari ekosistem musik modern. Pergeseran kolaborasi musik ke arah yang lebih digital memunculkan tantangan baru, yaitu bagaimana membuat ruang digital mempunyai *feel* yang sama dengan ketika kolaborasi musik yang fisik atau tradisional. Salah satu tantangan krusial yang dihadapi adalah koordinasi timing antar pemain dalam lingkungan digital yang rentan terhadap variasi latensi jaringan.

   Tantangan yang dihadapi pada kolaborasi musik secara *online* dan *real-time* berbeda dari platform kolaborasi teks seperti *Google Docs.* Karena toleransi latensi pada aplikasi *text editing* lebih besar dan bisa sampai sekian ratus *millisecond,* sedangkan persepsi orang yang terlatih secara musikal memiliki kepekaan yang luar biasa terhadap perbedaan *feel* waktu. Riset di bidang *Networked Music Performance (NMP)* juga menunjukkan bahwa latensi yang melampaui batas 20-30 *milliseconds* dapat mengganggu sinkronisitas dan menurunkan kualitas permainan ansambel (Lakiotakis et al., 2018). Bahkan kenaikan latensi yang kecil, atau *jitter,* dapat mengacaukan koordinasi antar pemain. Maka keberhasilan sistem kolaborasi musik *online* tidak hanya bergantung pada konektivitas jaringan yang cepat, tapi juga memastikan implementasi mekanisme sinkronisasi waktu dengan akurat untuk beroperasi dalam batasan manusia.

   Dalam praktiknya tantangan sinkronisasi *timing* antar pemusik ini berdampak nyata pada pengalaman kolaborasi musik. Latihan musik *in-person* mengandalkan isyarat visual (gerakan, kontak mata) untuk koordinasi *timing* dan ekspresi lagu, keterbatasan isyarat visual pada platform digital menyulitkan sinkronisasi terutama jika ada perubahan tempo atau bagian-bagian yang kompleks (Bishop et al., 2017). Isyarat visual krusial untuk sinkronisasi pada momen tertentu, seperti pada waktu masuk lagi, perubahan tempo, dan jika ada jeda

pada lagu. Jika tidak ada isyarat visual, musisi menunjukkan konsistensi *timing* yang kurang, terutama pada momen-momen penting pada lagu (Bishop et al., 2017). Ketidaksinkronan antar pemain juga bisa terjadi jika part lagu yang dimainkan secara *rhythmic* sulit dan ketat, sehingga membuat pemain susah untuk mengikuti permainan mereka sampai mana dan koordinasi dengan pemain lain, ditambah dengan *timing error* yang sifatnya individual seperti kurangnya konsentrasi dan kesalahan teknikal bisa menyebabkan ketidaksinkronan yang meskipun kecil namun juga dibiarkan bisa menyebabkan masalah (D'Amario et al., 2018). Kondisi ini menunjukkan bahwa meskipun teknologi kolaborasi musik telah berkembang, aspek sinkronisasi visual dan timing coordination masih menjadi area yang memerlukan penanganan lebih lanjut.

Adopsi partitur digital pada device individual dalam rehearsal modern membawa implikasi pada alur koordinasi. Dalam konteks tradisional dengan partitur kertas, seluruh ensemble memiliki referensi visual yang sama yang memudahkan koordinasi posisi (e.g., "mulai dari bar 32, ulangi intro"). Ketika setiap pemain menggunakan device terpisah tanpa mekanisme sinkronisasi, koordinasi ini mengandalkan komunikasi verbal dan manual navigation, yang berpotensi menimbulkan masalah terutama saat latihan. Meskipun belum terdapat evaluasi sistematis mengenai magnitude masalah ini, penelitian ini mengeksplorasi apakah *synchronized visual playback* dengan *cursor tracking* dapat mengurangi masalah teknis koordinasi dan meningkatkan efisiensi latihan dalam konteks penggunaan partitur digital. Pendekatan ini tidak menggantikan visual cues tradisional yang tetap krusial untuk koordinasi permainan bersama (Bishop et al., 2017).

Beberapa aplikasi terdahulu mencoba mengimplementasikan konsep kolaborasi secara *online*. *BandLab* dan *Soundtrap* membuat *Digital Audio Workstation (DAW)* yang bersifat cloud, memungkinkan masyarakat awam mencoba produksi musik menggunakan DAW yang lebih simpel. *JamKazam dan Jamulus* adalah *platform* kolaborasi juga, kedua aplikasi tersebut memanfaatkan audio dengan latensi rendah yang dikirim secara *peer-to-peer* memungkinkan pengguna bermain secara *real-time* melalui internet. Namun, pendekatan ini mengalami kendala teknis karena butuh jaringan yang stabil. Meskipun aplikasi-aplikasi ini telah menyediakan fitur kolaborasi *real-time*, evaluasi empiris terhadap akurasi sinkronisasi timing (khususnya dalam konteks *visual playback*) masih terbatas dalam literatur.

Penelitian oleh Buffa dan Burns (2025) mengeksplorasi kolaborasi musik berbasis web melalui Web Audio Modules (WAM) yang membuat pengguna bisa membuat musik secara interaktif dan real-time dari browser. Hasilnya menunjukkan bahwa platform web

mempunyai potensi besar dalam mendukung kolaborasi musik real-time, namun mekanisme sinkronisasi antar klien yang diterapkan masih mengandalkan latency compensation browser dan belum menggunakan model waktu terpusat berbasis server. Meskipun beberapa platform komersial seperti BandLab telah mengimplementasikan fitur kolaborasi *real-time* untuk MIDI dan partitur, belum terdapat studi empiris yang mengevaluasi seberapa efektif kombinasi server-authoritative clock dan client-side interpolation dalam mencapai target sinkronisasi < 30ms untuk konteks *music rehearsal* dengan *visual playback*. Selain itu, sebagian besar penelitian terdahulu berfokus pada sinkronisasi audio streaming (JamKazam, Jamulus) atau kolaborasi *asynchronous* (cloud DAW), sementara kebutuhan akan *synchronized visual cues* untuk koordinasi ensemble rehearsal belum mendapat perhatian yang memadai dalam literatur akademis.

Penelitian ini bertujuan untuk mengisi celah tersebut dengan melakukan evaluasi empiris terhadap implementasi *hybrid synchronization model* yang menggabungkan *server-authoritative clock* dan *client-side interpolation* dalam konteks partitur musik kolaboratif. Fokus penelitian adalah pada *same-room rehearsal scenario*, di mana musisi berlatih bersama dalam satu lokasi namun menggunakan device terpisah untuk akses partitur. Kontribusi utama penelitian ini adalah demonstrasi dan evaluasi kuantitatif terhadap achievable *synchronization accuracy* (target MAD ≤30ms) menggunakan kombinasi teknik *server-authoritative* dengan *periodic re-synchronization*, serta validasi persepsi pengguna terhadap kualitas sinkronisasi dalam konteks kolaborasi musik nyata.

Skripsi ini memadukan mekanisme *server-authoritative clock* dan *client interpolation* untuk mengimplementasikan sistem *real-time* yang sinkron dan efektif. *Server-authoritative clock* bertugas jadi sumber utama waktu, di mana klien menyesuaikan jam lokal berdasarkan waktu global server memastikan seluruh klien sinkron dengan satu sumber waktu. Pendekatan *server-authoritative* ini mengatasi *clock drift* dan *latency* yang tidak sama antar klien, karena semua aksi yang berhubungan dengan waktu berdasarkan server sebagai sumber global. Dalam konteks sinkronisasi waktu, konsep ini memastikan waktu yang digunakan di semua klien tidak fleksibel, karena ditentukan oleh satu sumber yang terpusat yang menjadi referensi utama (Mani et al., 2018).

Client-side interpolation adalah metode untuk menjaga kelancaran pengalaman aplikasi di sisi pengguna dengan memperkirakan posisi atau status waktu antar *update*. Artinya meskipun data dari server tiba secara tidak pasti atau mungkin dipengaruhi oleh gangguan jaringan, klien tetap dapat menampilkan pergerakan waktu atau animasi pemutaran secara halus melalui proses interpolasi lokal (Unity Technologies, 2024).

Kombinasi antara model waktu yang sifatnya *server-authoritative* dan visualisasi dari sisi klien yang didukung oleh perhitungan lokal ini diharapkan bisa membuat sistem sinkronisasi *real-time* yang stabil dan responsif, bahkan pada jaringan yang tidak ideal.

Konsep aplikasi ini terbagi menjadi dua bagian utama: deteksi chord dan kanvas partitur tempat pengguna dapat mengedit dan bermain bersama. Aplikasi menerima file audio dari pengguna, kemudian mendeteksi chord menggunakan *library chord-extractor*. Hasil deteksi berupa data chord dan posisi waktunya, yang kemudian ditampilkan dalam *layout* partitur. Bagian kedua mencakup dua mode: *Play Along* dan *Play Together*. Play Along, pengguna dapat bermain bersama partitur sambil mengikuti audio yang di-*upload*. Partitur hasil ekstraksi tidak dapat diubah agar tetap sinkron dengan audio asli. *Play Together*, pengguna bebas membuat partitur dari awal atau memakai hasil ekstraksi sebagai acuan, dan seluruh pengguna dalam satu proyek dapat *edit* partitur secara kolaboratif mirip *Google Docs*. Setiap room memiliki satu *leader* yang mengatur hak akses dan menjadi satu-satunya yang dapat memulai sesi bermain. Secara umum, kedua mode berbagi tujuan yang sama: menyediakan *playback* partitur yang sinkron di seluruh pengguna, sehingga dapat bermain bersama dengan waktu yang konsisten.

3. Rumusan Masalah

Berdasarkan latar belakang, berikut adalah rumusan masalah dalam penelitian ini:

1) Seberapa efektif kombinasi mekanisme *server-authoritative clock* dan *client-side interpolation* dalam menjaga konsistensi waktu *playback* antar-klien pada aplikasi partitur musik kolaboratif *real-time?*
2) Seberapa besar penyimpangan waktu rata-rata antar-klien yang dapat dicapai oleh sistem?
3) Bagaimana persepsi pengguna terhadap kualitas sinkronisasi dan kenyamanan kolaborasi dalam konteks musik real-time ketika menggunakan sistem?

4. Tujuan

Tujuan dari penelitian ini adalah mengembangkan dan menerapkan model sinkronisasi waktu berbasis *server-authoritative clock* dengan *client-side interpolation* untuk menjaga konsistensi waktu global pada aplikasi partitur musik kolaboratif *real-time*. Penelitian ini juga bertujuan mengukur seberapa besar penyimpangan waktu rata-rata antar-klien yang dapat dicapai oleh sistem sebagai indikator akurasi sinkronisasi. Selain itu, penelitian ini ingin memahami bagaimana persepsi pengguna terhadap kualitas sinkronisasi dan tingkat kenyamanan kolaborasi musik ketika menggunakan sistem yang dikembangkan.

5. Ruang Lingkup

Fokus penelitian adalah pada keakuratan sinkronisasi dan pengalaman pengguna terhadap sistem sinkronisasi yang dikembangkan.

1. Lingkup sistem berdasarkan fitur (Input-Proses-Output)
    a. Fitur Ekstraksi dan Penyusunan Partitur

    Input: Pengguna bisa *upload* file lagu ke dalam aplikasi yang nanti akan diproses oleh *script* pendeteksi chord. Secara default *script* python sudah bisa mendeteksi tempo dan *time signature* dari lagu, namun pengguna bisa memasukkan tempo dan *time signature* sendiri untuk mendukung keakuratan data.

    Proses: File lagu dari pengguna diterima oleh *script* dan dilakukan proses ekstrak chord dari lagu tersebut, dibantu oleh *package* python *chord-extractor*. Data hasil ekstrak chord ini akan didukung oleh *package* python librosa yang mendeteksi tempo, *downbeat,* dan *energy.* Data-data dari package *chord-extractor* dan librosa akan digunakan untuk mendeteksi struktur lagu dan menjadi referensi untuk meletakkan chord ke format partitur yang siap untuk dikirim ke *frontend*.

    Output: Hasil dari proses *script* python nanti adalah sebuah file yang berformat .json, berisi data untuk partitur seperti chord, tempo, dan *time signature*.

    b. Fitur Partitur Kolaboratif

    Input: Aksi pengguna dalam kanvas partitur termasuk: *edit chord, add/remove bar, select mode (Play Together/Play Along).* Di dalam ruang lingkup projek, pengguna bisa mengatur peran pengguna lain yang diundang, antara *leader* atau *member*.

    Proses: Setiap *event update* dari hasil *editing* pengguna terhadap partitur dicatat ke dalam server oleh *Firebase Real-Time Database*

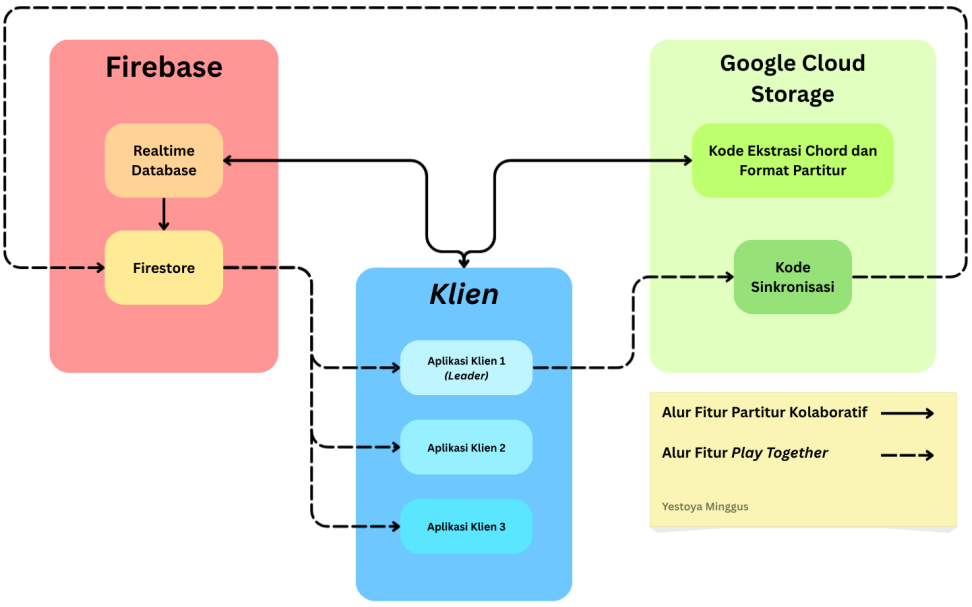
    Output: Perubahan pada partitur langsung tersinkron ke seluruh perangkat yang berada dalam proyek yang sama. Setiap klien menerima *update real-time* dan menampilkan partitur secara konsisten, termasuk *update* struktur bar, chord, pembaruan status mode dan peran pengguna dalam room. Hasilnya, seluruh pengguna melihat partitur yang sama tanpa konflik.

c. Fitur *Play Together* dan Sinkronisasi

Input: Aksi pengguna, termasuk perintah *play, pause*, *seek*, dan navigasi partitur. *Leader* sebagai pengendali utama dapat memulai atau menghentikan sesi, sementara *member* bisa menekan *"ready"* dan mengikuti partitur pada saat sesi berjalan.

Proses: Aksi *leader* dicatat sebagai event sinkronisasi dalam Firebase Realtime Database, kemudian server mengirimkan waktu global yang menjadi acuan bersama untuk mulai sesi. Klien menghitung offset terhadap waktu server dan melakukan client-side interpolation untuk menjaga pergerakan partitur tetap halus dan konsisten meskipun terdapat variasi latensi jaringan.

Output: Seluruh perangkat dalam sesi *Play Together* memutar partitur pada waktu yang sama, menampilkan *cursor* kemajuan *bar* dan *beat* secara sinkron.



Gambar 5.1 Blok Diagram Aristektur Sistem

2. Teknologi yang digunakan

Sistem menggunakan *Flutter* sebagai *frontend* di sisi pengguna, karena aplikasi bersifat *mobile.* Penggunaan *Flutter* didukung oleh penggunaan *Riverpod* sebagai *state management* supaya alur data mudah untuk dikontrol. *Backend* sistem menggunakan layanan *Firebase,* yaitu Firestore untuk menyimpan data proyek dan partitur, serta Realtime Database untuk event sinkronisasi yang butuh latensi rendah. File audio disimpan di *Google Cloud Storage* dan diproses oleh sebuah *script* berbasis

Python yang berjalan di Cloud Run, menggunakan library *chord-extractor* untuk mengambil chord dari audio. Seluruh arsitektur dirancang serverless agar ringan, mudah di-*deploy*, dan tidak perlu mengurus server manual.

3. Batasan penelitian dan sistem
    a. Sistem ini dikembangkan berbasis aplikasi dengan *frontend* menggunakan Flutter, *backend* menggunakan *Firebase (Firestore & Real-time Database),* sistem berbasis Python yang dimasukkan ke dalam *container* Docker untuk kemudian *deploy* ke Google Cloud Run.
    b. Berfokus hanya pada evaluasi implementasi mekanisme *server-authoritative clock* dan *client interpolation* untuk mendukung sistem aplikasi musik yang *real-time.*
    c. Tidak berfokus pada akurasi ekstraksi chord atau fitur *editing* partitur yang di luar ruang lingkup.
    d. Aplikasi ditujukan pada pengguna yang berlatih bersama pengguna lain di tempat yang sama, bukan berbeda lokasi.
    e. Sistem tidak melakukan sinkronisasi audio langsung antar pengguna, bukan sinkronisasi audio, melainkan sinkronisasi partitur.
4. Rencana Pengujian

    Pengujian pada sistem ini dilakukan untuk melihat seberapa baik model sinkronisasi bekerja dan bagaimana pengalaman pengguna ketika berkolaborasi. Dari sisi teknis, sistem akan diuji menggunakan pengukuran penyimpangan waktu antar-klien dengan menghitung *Mean Absolute Deviation (MAD)*. Nilai ini digunakan sebagai indikator selisih antar-klien, terutama saat memutar partitur secara bersamaan. Pengujian teknis akan dilakukan di dua kondisi yang berbeda, 2-3 klien pada di ruangan yang sama dan pada lokasi geografis yang berbeda (jarak > 1km). Setiap pengujian teknis juga akan dilakukan dua kali, pertama dengan tidak menggunakan mekanisme resinkronisasi dan kedua menggunakan resinkronisasi, tujuannya adalah untuk mengevaluasi apakah resinkronisasi periodik diperlukan pada kondisi ini dan membandingkan performa antara *same-room* dan *cross-location.*

    Pemilihan MAD dilakukan karena metrik ini lebih stabil dan mudah dibaca untuk konteks sinkronisasi *real-time* dalam konteks penelitian ini karena MAD menghitung *spread* dari data *(client-to-client).* Variasi jaringan kadang menimbulkan lonjakan error kecil yang munculnya tidak teratur, dan metrik seperti RMSE biasanya

membuat lonjakan seperti itu terlihat lebih besar dari kondisi sebenarnya, atau MAE yang lebih mengukur antara perhitungan prediksi dan sebenarnya *(client-to-server)*. MAD hanya menghitung rata-rata selisih absolut, sehingga hasilnya lebih representatif terhadap performa sinkronisasi. Selain itu, nilai MAD yang langsung dalam milidetik membuatnya lebih mudah dihubungkan dengan batas persepsi musisi terhadap keterlambatan ritme.

Selain pengujian teknis, penelitian ini juga melibatkan kuesioner kepada pengguna yang mencoba fitur *Play Together.* Kuesioner ini fokus pada bagaimana mereka merasakan kualitas sinkronisasi, apakah ada jeda yang mengganggu alur bermain, dan bagaimana kenyamanan mereka selama sesi kolaborasi. Kuesioner juga akan membandingkan hasil latihan dengan aplikasi dengan metode tradisional, sehingga mengevaluasi apakah kebutuhan dan apakah aplikasi menyelesaikan masalah sinkronisasi musisi dalam permainan bersama. Gabungan antara hasil metrik MAD dan persepsi pengguna diharapkan memberi gambaran yang lebih utuh tentang performa sinkronisasi yang diterapkan pada aplikasi.

6. Manfaat Penelitian

Penelitian ini diharapkan memberikan kontribusi akademis melalui penerapan mekanisme *server-authoritative clock* dan *client-side interpolation* sebagai solusi sinkronisasi waktu pada sistem kolaborasi real-time. Dengan pendekatan tersebut, penelitian ini juga menghasilkan prototipe aplikasi partitur musik kolaboratif yang mampu mempertahankan keseragaman waktu. Aplikasi ini juga menyediakan sarana latihan digital bagi musisi untuk berkolaborasi tanpa hambatan waktu, memberikan pengalaman bermusik yang lebih natural. Secara umum, hasil penelitian ini diharapkan dapat mendukung pengembangan studi lanjutan di bidang teknologi musik dan sistem sinkronisasi *real-time*, khususnya dalam konteks implementasi arsitektur terdistribusi yang responsif dan presisi di lingkungan akademik.

7. Landasan Teori

a. Konsep Partitur dan Representasi Musik



Gambar 7.1 Partitur untuk lagu *"Spain"* oleh Chick Corea

Partitur musik (Gambar 7.1) adalah representasi tertulis atau cetak dari sebuah komposisi musik yang menunjukkan semua bagian atau instrumen yang terlibat, biasanya menggunakan notasi musik standar pada paranada. Partitur ini menampilkan nada, ritme, dinamika, dan elemen musik lainnya di beberapa paranada yang disejajarkan secara vertikal untuk menunjukkan bagaimana bagian-bagian tersebut berkoordinasi dan saling terkait (Britannica, 2017). Partitur digunakan oleh musisi untuk membaca bagian instrumen dan mengikuti tempo serta *rhythm* agar sinkron dengan pemain yang lain. Dalam penelitian ini partitur digunakan oleh pengguna untuk membaca *chord* yang sedang dimainkan, untuk mencapai sinkronisasi antar pemain *cursor* partur pada saat *playback* juga harus sama dengan *chord* yang dimainkan dan harus sama di seluruh klien.

b. Networked Music Performance (NMP)

Networked Music Performance adalah praktik bermusik secara bersama melalui jaringan komputer yang dilakukan secara *real-time*, atau mengutip dari Gabrielli, *"the practice of conducting real-time music interaction over a computer network"* (MUTOR, 2021). Tujuan dari NMP adalah memungkinkan musisi berkolaborasi dari lokasi berbeda secara sinkron melalui koneksi jaringan. Namun bidang NMP ini masih menemukan tantangan yang besar, yaitu *audio delay (latency, network delay), audio dropout,* dan sinkronisasi. Faktor-faktor tersebut menjadi gangguan yang paling kritis karena bisa mempengaruhi sinkronisasi *rhythm* antar musisi.

Tantangan sinkronisasi ini sangat penting, karena musisi memiliki kepekaan terhadap *rhythm* yang sangat tinggi. Batas ini, atau disebut dengan *Just Noticeable Difference (JND),* terbukti bisa serendah 20 hingga 30 *milliseconds* (Schmid et al., 2024). Riset di bidang *Networked Music Performance (NMP)* secara konsisten menunjukkan bahwa latensi yang melampaui batas 20 hingga 30 *milliseconds* dapat mengganggu sinkronisitas dan menurunkan kualitas permainan ansambel (Lakiotakis et al., 2018). Penelitian ini mengadopsi prinsip dasar dari NMP yaitu kolaborasi musik jarak jauh, tapi lebih fokus kepada sinkronisasi waktu yang simbolik melalui partitur, bukan audio. Dengan kata lain, penelitian ini menggunakan arsitektur internet *real-time* seperti NMP, tetapi pengujian dilakukan pada lingkungan lokal (misalnya jaringan wifi yang sama) untuk mengisolasi variabel teknis dan memperoleh data sinkronisasi yang lebih terukur. Pendekatan ini memungkinkan evaluasi performa waktu tanpa dipengaruhi oleh perbedaan kualitas jaringan antar wilayah, yang sering kali menjadi faktor eksternal pada NMP.

c. Music Information Retrieval (MIR)

Music Information Retrieval atau MIR adalah ilmu dan teknologi untuk mengekstraksi, menganalisis, dan mengambil informasi yang bermakna dari data musik, terutama dari rekaman audio (ScienceDirect, 2025). Jenis data yang diekstrak bisa berupa tempo, *chord*, *beat, onset,* dan bisa juga struktur lagu. Dalam penelitian ini teknologi MIR digunakan sebagai alat bantu, atau fitur untuk mengekstraksi data dari audio. Data yang diekstrak ini akan ditampilkan sebagai partitur digital dalam aplikasi dan akan digunakan untuk fitur sinkronisasi visual partitur dan *playback metronome*.

Proses ekstraksi informasi dari audio dilakukan oleh kode *script* yang ditulis dalam Python, dan menggunakan beberapa *package* pembantu:

- **Librosa:** digunakan untuk analisis *Root Mean Square* (RMS) *energy* dari audio, deteksi *downbeats,* dan melakukan separasi audio menggunakan *Harmonic-Percussive Source Separation* (HPSS).
- **chord-extractor:** merupakan *wrapper* dari ekstensi Chordino yang merupakan *plugin* dari aplikasi DAW Vamp. *Package* ini menganalisis dan mengekstraksi *chord* dan *timestamp* nya dalam audio.

Semua informasi yang didapat dari kedua *package* tersebut akan digunakan untuk menyusun hasil *chord* ke format yang lebih teratur dalam partitur, lalu mengirimkannya dalam bentuk JSON ke dalam aplikasi supaya bisa ditampilkan.

d. Teori Dasar Sinkronisasi Waktu

Di dalam sistem yang terdistribusi, setiap *device* yang terhubung pasti punya waktu lokal yang berbeda-beda (Lall et al., 2023), perbedaan ini bisa jadi karena faktor *hardware* ataupun *operation system* nya. Penyebab lain yang bisa menyebabkan ketidaksinkronan waktu adalah jaringan, jaringan yang tidak stabil bisa menyebabkan *delay* saat pengiriman pesan (Liu et al., 2019). Ada beberapa pendekatan umum yang sudah dikembangkan dalam menangani masalah sinkronisasi waktu ini, yaitu:

- **Model *Client-Server:*** Dalam model ini *server* menjadi pusat dari segalanya, dari *resource,* data, hingga *service*. Lalu banyak pengguna atau klien bisa terhubung ke *server* untuk memakai *service* ini. Kelebihan dari model ini adalah sifatnya yang terpusat, mudah dikelola dan lebih aman karena data ada di satu tempat. Kekurangan dari model ini adalah jika *server* mati, maka seluruh layanan akan mati juga *(single point of failure),* model ini juga rawan terhadap *bottleneck* karena terlalu banyak pengguna yang terhubung (Apriadi et al., 2025).
- **Model *Peer-to-Peer:*** Sistem ini sifatnya terdesentralisasi, artinya tidak ada yang menjadi *server* pusat. Semua yang terhubung dalam jaringan memiliki kedudukan yang sama dan bisa bertindak sebagai “klien” yang meminta data dan juga “server” yang mengirim data secara bersamaan. Kelebihan dari model ini adalah sangat kokoh, jika satu *peer* mati jaringan akan tetap berjalan, model ini juga efisien dalam pengelolaan data yang besar karena terbagi-bagi. Kekurangan dari model ini adalah konfigurasi dan pengelolaan

keamanan yang sulit, karena harus memverifikasi siapa saja yang bisa dipercaya (Apriadi et al., 2025).

Penelitian ini akan menggunakan model *server-authoritative* yang merupakan sub bagian dari model *client-server*. Model *server-authoritative* lebih cocok untuk aplikasi *real-time* yang membutuhkan sinkronisasi akurat (López et al., 2020) seperti musik, karena hanya ada satu sumber kebenaran waktu untuk mencegah konflik antar klien.

Salah satu pendekatan umum yang digunakan dalam sistem sinkronisasi waktu adalah algoritma Cristian atau *Network Time Protocol (NTP)* (Shenoy, 2019). Konsep dasarnya adalah pertukaran empat penanda waktu antara klien dan server untuk menghitung dua nilai utama, yaitu *offset* dan *delay* (JumpCloud, 2025). Empat waktu tersebut adalah:

- ***t1:*** waktu lokal saat klien mengirim permintaan
- ***t2:*** waktu *server* saat permintaan diterima
- ***t3:*** waktu server saat mengirim *response*
- ***t4:*** waktu lokal saat menerima *response server*

Berdasarkan keempat nilai itu, dapat dihitung:

$$offset = \frac{(t2-t1)+(t3-t4)}{2} \text{ dan } delay = (t4 - t1) - (t3 - t2)$$

Offset menghitung selisih waktu antar waktu klien dan waktu global milik *server,* sedangkan *delay* menggambarkan waktu *round-trip* dari pesan. Dalam penelitian ini, proses perhitungan tersebut tidak dilakukan secara manual, karena *Firebase Real-time Database* sudah disediakan variabel *".info/serverTimeOffset"* yang berisi nilai estimasi *offset* dari waktu lokal dan waktu *server Firebase.* Nilai ini akan *update* secara otomatis karena SDK *Firebase* menggunakan mekanisme dua-arah yang sama dengan perhitungan dua rumus di atas. Sistem nanti akan memperoleh estimasi waktu *server* secara langsung melalui:

$$estimatedServerTime = DateTime.now() + serverTimeOffset$$

Pendekatan ini memungkinkan aplikasi untuk memiliki waktu global yang konsisten, sekaligus mempertahankan sifat yang *server-authoritative* di seluruh klien.

Untuk mencapai sinkronisasi presisi tinggi, sistem ini menerapkan model server-authoritative clock yang menjadikan waktu server sebagai satu-satunya sumber kebenaran. Mekanisme ini dikombinasikan dengan pendekatan *Server-Coordinated Scheduled Start*, di mana server menentukan waktu mulai absolut beberapa milidetik ke depan dan seluruh klien melakukan pemutaran tepat pada waktu tersebut. Dengan cara ini, waktu mulai tidak lagi bergantung pada waktu

komit data di server, tetapi pada waktu absolut yang terkoordinasi, sehingga akurasi antar-klien dapat dijaga di bawah 30 ms. Selain itu, penelitian juga ini mengacu pada konsep dasar sinkronisasi:

- ***Client-side interpolation:*** menjaga kelancaran *update* di sisi klien

$$t_{local} + \alpha \times (t_{server} - t_{local})$$

- ***Mean Absolute Deviation:*** digunakan untuk evaluasi waktu antar klien selama sinkronisasi

$$\frac{\Sigma|positionClient_i(t) - positionMean(t)|}{N}$$

Nilai MAD yang semakin kecil menunjukkan sistem sinkronisasi berjalan semakin akurat, dengan target deviasi di bawah 30 ms agar tetap dalam batas persepsi musikal manusia (Schmid et al., 2024; Lakiotakis et al., 2018).

e. Server-Authoritative Clock

Server authoritative, adalah suatu model *network* dimana *server* pusat mempunyai otoritas atas seluruh sistem, artinya *server* menjadi satu-satunya sumber kebenaran untuk data, *decision making,* dan waktu *(single source of truth)* (Trainor, 2022). Biasanya model ini digunakan dalam *game online* yang *multiplayer* untuk mencegah *cheating* dan menyamakan *game state*, namun dalam penelitian ini model digunakan untuk menyamakan waktu partitur setiap klien. *Server* mengirimkan *timestamp* global kepada klien secara periodik lalu klien menerima variabel *serverOffset* untuk menyesuaikan waktu lokal dari *device*. Klien tidak bisa mengubah *timestamp* global ini karena merupakan waktu *server*, klien hanya bisa menerima, menyesuaikan, dan menginterpolasi dengan waktu lokal agar tetap sinkron dengan *server.* Pemilihan model ini dilakukan untuk menghindari konflik sinkronisasi antar klien dan memastikan konsistensi waktu yang global.

f. Client-Side Interpolation

Client-side interpolation adalah teknik untuk memperkirakan posisi atau nilai di antara dua titik waktu berdasarkan waktu server (Bernier, 2008). Umumnya digunakan di sistem *real-time* untuk menjaga kelancaran visual atau pergerakan meski ada *delay* dari jaringan. Teknik ini mengurangi efek *lag* dan *jitter* data akibat jaringan dan membuat pengalaman pengguna terlihat sinkron dan halus. *Server* mengirimkan *update* data waktu secara periodik lalu klien menghitung posisi antara dua *update* berdasarkan waktu lokal. Hasilnya pergerakan atau *event* di sisi klien terasa halus dan tidak “loncat” ke posisi tertentu. Dalam penelitian ini *client-side*

interpolation sangat penting karena berguna untuk menyamakan posisi *metronome* dan *cursor* partitur terhadap waktu server, teknik ini menjaga kehalusan visual untuk menjaga *rhythm* pengguna saat bermain bersama-sama.

8. Aplikasi dan Penelitian Terdahulu

Tabel *State of the Art*

Aspek	BandLab	JamKazam	Buffa & Burns sequencer.party / WAM	Penelitian ini
Masalah yang Diselesaikan	Tidak ada sinkronisasi *real-time playback*; kolaborasi hanya berbasis manual save	Latensi jaringan tinggi & jitter membuat permainan *real-time* sulit	Sinkronisasi klien web belum kuat; masih mengandalkan kompensasi *latency browser*	Sinkronisasi waktu antar-klien untuk *playback* partitur musik; *jitter* pada data *real-time*
Metode / Pendekatan Teknis	*DAW cloud*, sinkronisasi berbasis file & manual commit	Streaming audio P2P latensi rendah	Web Audio Modules (WAM) untuk kolaborasi musik real-time di browser	Model *server-authoritative clock + client-side interpolation, Firebase RTDB + Firestore*
Hasil / Keluaran Sistem	Kolaborasi tidak *real-time*, tidak mendukung *playback* sinkron antar pengguna	Tergantung jaringan stabil, sering terjadi ketidaksinkronan ritme	Potensial besar tapi sinkronisasi waktu antar klien belum terpusat dan belum stabil	*Playback* dan *cursor* sinkron *jitter* dihaluskan akurasi diuji dengan MAD fokus pada sinkronisasi data, bukan audio

9. Metodologi Penelitian

a. Langkah Pengerjaan

1. Studi Literatur

   Tahap pertama dalam penelitian ini adalah studi literatur, dimana akan dilakukan proses pengumpulan referensi dan teori yang akan digunakan sebagai dasar penelitian:

   - **Konsep sinkronisasi waktu*:*** Mempelajari model *server-authoritative clock,* menggunakan *offset* sebagai sumber sinkronisasi, dan mengatasi *network delay* serta *jitter.*
   - **Teknik *Client-side Interpolation:*** Mempelajari lebih mendalam mengenai teknik umum yang digunakan pada *game online* dan

sistem kolaboratif *real-time* dan bagaimana implementasinya pada aplikasi musik.

- **Studi Arsitektur:** Mempelajari lebih mendalam dokumentasi mengenai Firebase yang digunakan sebagai backend dan mesin utama dengan menggunakan *Real-time Database.* Mempelajari implementasi Google Cloud Run dan Docker yang nanti digunakan untuk *deploy script* ekstraksi audio.

Hasil dari studi literatur ini akan menjadi dasar untuk merancang sistem arsitektur dari aplikasi agar efisien dan dapat diuji secara kuantitatif.

2. Perancangan dan Desain Sistem

Arsitektur Sistem

Sistem aplikasi dirancang menggunakan arsitektur dengan empat komponen:

- **Frontend** menggunakan Flutter, mendukung *development* untuk perangkat *mobile* yang *cross-platform*. Flutter juga didukung oleh package dan function yang cocok untuk aplikasi yang berbasis *real-time*.
- **Firestore & *Real-time Database*** digunakan sebagai mesin utama dari sinkronisasi *real-time* dan penyimpanan status global.
- ***Script* Python** untuk menjalankan kode ekstrak audio dan *script* singkat yang mengirim *trigger* waktu mulai *playback* ke *Firestore*, python yang dimasukkan ke dalam Docker lalu *deploye* melalui Google Cloud Run.
- **API Gateway** untuk autentikasi dan data user.

Desain Model Data Firestore

Fitur kanvas partitur yang bisa diedit bersama dalam aplikasi ini akan menggunakan Firestore, dimana model data dari Firestore adalah NoSQL. Ada *collection* utama *"/sessions"* yang menyimpan status global dari projek tersebut, seperti tempo, jumlah bar, dan *state* dari *playback.* Lalu di dalamnya ada *sub-collection "/sheets"* dan *"/participants"* yang menyimpan data spesifik partitur dan pengguna seperti *chord, role,* dan *notes.* Desain NoSQL diharapkan dapat memudahkan keseragaman dalam dokumen dan menghindari konflik ada saat *editing* partitur, serta memudahkan sinkronisasi pada saat *playback* dimainkan.

Konsistensi Data dan Konflik Penulisan

Sistem kolaborasi partitur dalam penelitian menerapkan strategi *Last-Write-Wins (LWW)* sebagai mekanisme resolusi konflik. Pemilihan strategi ini dikarenakan komponen partitur untuk *chord* cara kerja nya hanya ada satu slot *textfield* yang bisa diisi, misalnya untuk setengah bar ada satu 2 komponen slot *chord* yang bisa diisi, per komponen hanya bisa diisi satu *chord*, sehingga kemungkinan konflik *edit* bersamaan sangat kecil. Dalam strategi LWW, setiap perubahan ke *server* diberi *timestamp* dari waktu *server.* Ketika dua *update* masuk untuk slot data yang sama dan datang di waktu yang hampir sama, maka yang sistem hanya mempertahankan data yang memiliki *timestamp* paling baru. Strategi LWW cocok diimplementasikan untuk penelitian ini karena jenis komponen partitur yang mendukung dan sejalan dengan mekanisme utama penelitian ini yaitu *server-authoritative.* Strategi ini juga lebih sederhana, dibandingkan strategi yang kompleks seperti *Operational Transform (OT)* atau *Conflict-Free-Replicated Data Type (CRDT)* karena karakteristik aplikasi tidak memerlukan penggabungan *update* yang bersamaan dalam dokumen yang besar seperti *Google Docs.*

Model Sinkronisasi Waktu

Sistem menerapkan model *server-authoritative clock* dengan sumber waktu global yang berasal dari *Firebase Real-time Database. Firebase* sudah menyediakan variabel *".info/serverTimeOffset",* yaitu estimasi selisih waktu antara waktu lokal dan waktu *server Firebase.*

Estimasi waktu *server* di sisi klien dihitung menggunakan rumus:

$$estimatedServerTime = DateTime.now() + serverTimeOffset$$

Nilai *offset* akan *update* setiap beberapa detik untuk menjaga konsistensi, menghindari ketidakkonsistenan jaringan. *Client* menggunakan nilai ini sebagai dasar perhitungan waktu mulai sinkronisasi.

Dalam sesi projek, hanya ada satu pengguna yang berperan sebagai *"Leader". Leader* akan mengirimkan perintah *Play* ke server, yang nanti akan menentukan waktu global *serverStartTime* yang ditambahkan beberapa ratus *miliseconds* ke depan untuk memberi waktu bagi semua klien melakukan sinkronisasi. Mekanisme ini menjaga seluruh klien untuk mulai *playback* di waktu yang sama, melalui sumber waktu yang tunggal.

Desain Alur Sinkronisasi Waktu

Fitur *"Play Together"* ini dimulai dengan *role "Leader"* yang menekan tombol *Play* dan menunggu semua pengguna untuk menekan *"Ready",* pada saat itu juga waktu *serverStartTime* akan direkam oleh *server* menggunakan *FieldValue.serverTimestamp()* agar semua klien memiliki referensi waktu yang sama. Namun metode ini memiliki kelemahan, karena waktu *commit Firestore* dapat berubah-ubah tergantung kondisi jaringan. Untuk memastikan sinkronisasi yang lebih akurat, penelitian ini menggunakan pendekatan *Server Coordinated Scheduled Start.* Saat *Leader* tekan *Play,* klien akan mengirimkan perintah ke *server,* lalu *server* akan menentukan waktu mulai yang absolut:

$$serverStartTime = currentServerTime + \Delta_{buffer}$$

di mana Δ_{buffer} adalah celah jeda untuk sinkronisasi (500-1000 ms). Nilai *serverStartTime* ini ditulis oleh *server* ke variabel *playbackState* di *Firestore* dan juga dikirimkan ke seluruh klien. Setiap klien akan menghitung mundur sebelum mulai:

$$starTime = serverStartTime - (localTime + serverTimeOffset)$$

Ketika mencapai nol maka setiap klien akan otomatis memulai *playback* secara bersamaan. Posisi kursor *playback* partitur akan dihitung secara berkelanjutan pada masing-masing klien menggunakan:

$$currentPositionMs = estimatedServerTime - serverStartTime$$

Setiap klien menggunakan nilai *currentPositionMs* tersebut untuk menyesuaikan posisi kursor partitur atau metronom secara *real-time*. Untuk Mencegah loncatan visual akibat jaringan, diterapkan teknik *client-side interpolation:*

$$t_{baru} = t_{local} + \alpha \times (t_{server} - t_{local})$$

Untuk mencegah *clock drift,* akan dilakukan resinkronisasi lagi bersama server dan mengambil nilai *offset* yang baru. Proses resinkronisasi ini dilakukan juga dengan menggunakan *client-side interpolation* Pendekatan ini akan menghindari ketidakkonsistenan akibat variasi jaringan dan memungkinkan semua klien mencapai sinkronisasi. Proses *"Syncing"* akan ditampilkan di *User Interface (UI)* pengguna selama masa sinkronisasi berjalan sebelum *playback* dimulai.

3. Pembuatan dan Implementasi Sistem

Pada penelitian pembuatan dan implementasi ini, desain dan rancangan yang ditetapkan di tahap sebelumnya akan direalisasikan. Langkah pertama adalah membuat *user interface* menggunakan Flutter, pengerjaan ini akan dilakukan dengan kode yang sifatnya dinamis mengingat nanti akan disambungkan dengan sistem *backend.* Lalu berikutnya dilakukan pengerjaan kode deteksi chord dengan python, menggunakan *package-package* MIR untuk ekstraksi audio. Kode python ini nanti akan dimasukkan ke dalam Docker *container* lalu *deploy* menggunakan Google Cloud Run.

Selanjutnya sistem *backend real-time*, membuat projek di Firebase, *setup collection* di *Firestore,* dan konfigurasi *Real-time Database* untuk fitur sinkronisasi dan kanvas *editing.* Setelah Firebase sudah selesai, akan dilakukan integrasi keseluruhan sistem ke dalam ekosistem Flutter. Pertama menyambungkan *script,* memastikan data JSON yang dikirim ke Flutter bisa ditampilkan dalam bentuk partitur digital. Lanjut ke fitur kanvas *editing,* menghubungkan ke *Firestore*, membuat sistem projek *room* dan *role* diantara pengguna. Terakhir, membuat fitur bermain bersama dengan partitur yang sinkron, menghubungkan Flutter dengan *Real-time Database*. Mengimplementasikan mekanisme *server-authoritative clock* dari *server* dan menerimanya ke dalam Flutter, lalu menggunakan *client-side interpolation* untuk membuat kursor *playback* jadi halus secara visual.

4. Pengujian Sistem

Dalam penelitian ini, pengujian sistem aplikasi akan dibagi menjadi dua kategori utama:

a. Pengujian Teknis

Pengujian dilakukan untuk mengukur tingkat akurasi sinkronisasi waktu antar-klien pada sistem dengan model *server-authoritative clock* dan mekanisme *Server-Coordinated Scheduled Start*. Tujuan pengujian adalah memastikan deviasi waktu antar-klien tetap berada di bawah 30 ms, sesuai batas persepsi musikal manusia terhadap perbedaan tempo (Schmid et al., 2024). Eksperimen pengujian dilakukan dengan 2-3 klien yang terhubung ke sistem, dilakukan pada beberapa skenario:

- *Same-room*: kondisi paling ideal dengan latensi rendah, semua klien berada di ruangan yang sama.
- *Cross-location*: kondisi dengan potensi latensi lebih tinggi, klien berada di lokasi geografis yang berbeda (jarak > 1km).

Semua klien menggunakan Android agar variabel lain tidak terlalu mempengaruhi hasil.

Skenario pengujian dilakukan 3-5 menit per trial, dilakukan dalam dua jenis dengan resinkronisasi dan tidak dengan resinkronisasi:

- *Baseline test*: 2-3 klien, 5 menit *playback* dengan WIFI dan dengan tidak menggunakan resinkronisasi. Menghitung MAD di 1 menit, 2 menit, 3 menit, 4 menit, dan 5 menit. Tujuannya untuk mengetahui apakah mekanisme resinkronisasi benar-benar dibutuhkan.
- *Stress test:* menguji kondisi internet dengan latensi yang tinggi, menguji 2-3 klien menggunakan 4G/5G. Menghitung MAD di interval yang sama, tujuannya sama dengan tes sebelumnya.
- *Cross-location:* menguji kondisi yang paling jelek, dimana klien dipisahkan oleh lokasi geografis berbeda dan berada di jaringan yang berbeda. Mengukur MAD di interval yang sama.

Pengujian teknis tersebut bertujuan untuk mengetahui apakah masalah *clock drift* benar-benar terjadi dan mekanisme resinkronisasi diperlukan. Jika hasil perhitungan pengujian menunjukkan *spike* pada MAD setelah menit tertentu, maka mekanisme resinkronisasi ini akan diimplementasikan ke dalam sistem. Setiap percobaan akan dilakukan *logging* pada masing-masing klien setiap 500ms. Semua log ini akan dikirim ke *Firestore* pada *sub-collection "/logs"* untuk analisis selanjutnya.

Tingkat keseragaman sinkronisasi diukur menggunakan *Mean Absolute Deviation (MAD)* antar klien:

$$\frac{\Sigma|positionClient_i(t) - positionMean|}{N}$$

dengan $timestampClient_i$ adalah waktu *event* yang catat klien ke-i, dan $timestampServer_i$ adalah waktu *server* saat *event* yang sama. Sistem dianggap berhasil apabila nilai MAD $\leq$ 30 ms.

b. Pengujian Persepsi Pengguna

Pengujian melibatkan 2-3 kelompok musisi yang berisi 2-3 orang per kelompok. Pertama musisi akan bermain bersama dengan fitur *playback* menggunakan *ear-monitor* dengan *clicker* metronome yang berjalan, hasil permainan ini akan direkam untuk nanti dievaluasi hasil permainan bersama. Kedua musisi akan bermain bersama lagi, namun tidak menggunakan *ear-monitor* untuk menilai langsung permainan bersama. Setelah kedua tahap tersebut, musisi akan diberikan kuesioner menanyakan tingkat kenyamanan dan pengalaman *delay* untuk mengevaluasi *feel* rhythm dan sinkronisasi dari sistem. Kuesioner juga akan membandingkan hasil latihan dengan aplikasi dengan metode tradisional, sehingga mengevaluasi apakah kebutuhan dan apakah aplikasi menyelesaikan masalah sinkronisasi musisi dalam permainan bersama.

5. Pembuatan Laporan

Setelah semua pengujian selesai, maka semua hasil akan didokumentasikan ke dalam laporan skripsi.

10. Jadwal Pengerjaan

Kegiatan	Bulan 1				Bulan 2				Bulan 3				Bulan 4			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
Studi Literatur	■	■														
Perancangan dan Desain Sistem		■	■	■	■	■										
Implementasi Sistem					■	■	■	■	■	■	■	■				
Pengujian Sistem											■	■	■	■		
Pembuatan Laporan														■	■	■

11. Daftar Referensi

- Buffa, M., & Burns, T. (2025, July). Real-Time Collaborative Music Creation on the Web: exploiting Web Audio Modules for Interactive Performance and Composition. In *30th IEEE Symposium on Computers and Communications (ISCC), workshop Next-Generation Multimedia Services at the Edge (NGMSE): Leveraging 5G and Beyond.* *https://dx.doi.org/10.5281/zenodo.10825647*
- Bishop, L., & Goebl, W. (2017). Beating time: How ensemble musicians' cueing gestures communicate beat position and tempo. Psychology of Music, 46, 84 - 106. https://doi.org/10.1177/0305735617702971.
- D'Amario, S., Daffern, H., & Bailes, F. (2018). Synchronization in Singing Duo Performances: The Roles of Visual Contact and Leadership Instruction. Frontiers in Psychology, 9. https://doi.org/10.3389/fpsyg.2018.01208.
- Elsevier. (n.d.). Music information retrieval. ScienceDirect. Diakses pada 5 November 2025, dari https://www.sciencedirect.com/topics/computer-science/music-information-retrieval
- Girela-López, F., López-Jiménez, J., Jiménez-López, M., Rodríguez, R., Ros, E., & Díaz, J. (2020). IEEE 1588 High Accuracy Default Profile: Applications and Challenges. IEEE Access, 8, 45211-45220. https://doi.org/10.1109/access.2020.2978337.
- JumpCloud. (2025). What is Network Time Protocol (NTP)? JumpCloud. Diakses pada 2 November 2025, dari https://jumpcloud.com/it-index/what-is-network-time-protocol-ntp
- Lakiotakis, E., Liaskos, C., & Dimitropoulos, X. (2018). Improving networked music performance systems using application-network collaboration. Concurrency and Computation: Practice and Experience, 31. https://doi.org/10.1002/cpe.4730.
- Lall, S., Caşcaval, C., Izzard, M., & Spalink, T. (2023). Logical Synchrony and the Bittide Mechanism. IEEE Transactions on Parallel and Distributed Systems, 35, 1936-1948. https://doi.org/10.1109/tpds.2024.3444739.
- Liu, K., Selivanov, A., & Fridman, E. (2019). Survey on time-delay approach to networked control. Annu. Rev. Control., 48, 57-79. https://doi.org/10.1016/j.arcontrol.2019.06.005.
- Mani, S., Durairajan, R., Barford, P., & Sommers, J. (2018). A System for Clock Synchronization in an Internet of Things. ArXiv, abs/1806.02474.

- Music Technology Online Repository. (n.d.). Unit 6: Networked music performance. History and Practice of Multimedia. Diakses pada 5 November 2025, dari https://mutor-2.github.io/HistoryAndPracticeOfMultimedia/units/06
- NPR. (2025, March 15). COVID five years later: How the pandemic changed music. NPR. https://www.npr.org/2025/03/15/nx-s1-5321053/covid-five-years-later-how-the-pandemic-changed-music
- Onderdijk, K., Acar, F., & Van Dyck, E. (2021). Impact of Lockdown Measures on Joint Music Making: Playing Online and Physically Together. Frontiers in Psychology, 12. https://doi.org/10.3389/fpsyg.2021.642713.
- Rizki Apriadi, Ridho Alvadil, Rena Zuriani Putri, & Herawati. (2025). IMPLEMENTATION AND ANALYSIS OF PEER TO PEER AND CLIENT SERVER NETWORKS USING CISCO PACKET. Digital Frontier: Journal of Computer and Science Innovation, 1(2).
- Schmid, A., Ambros, M., Bogon, J., & Wimmer, R. (2024). Measuring the Just Noticeable Difference for Audio Latency. Proceedings of the 19th International Audio Mostly Conference: Explorations in Sonic Cultures. https://doi.org/10.1145/3678299.3678331
- Shenoy, P. (2019). Lecture 12: Time ordering and clock synchronization [Catatan kuliah]. CMPSCI 677: Operating Systems, University of Massachusetts Amherst. https://lass.cs.umass.edu/~shenoy/courses/spring19/lectures/Lec12_notes.pdf
- The Editors of Encyclopaedia Britannica (2017, September 13). score. Encyclopedia Britannica. https://www.britannica.com/art/score-music
- Trainor, J. (2022). Single Source of the Truth in Media Data Centers. SMPTE Motion Imaging Journal, 131, 23-29. https://doi.org/10.5594/jmi.2022.3169748.
- Unity Technologies. (n.d.). Improving performance with client-side interpolation. Unity Manual. Diakses pada 30 Oktober 2025, dari https://docs.unity3d.com/Packages/com.unity.netcode.gameobjects@2.5/manual/learn/clientside-interpolation.html